

Eco-Guardian

Community-Driven Environmental Tracker

Project Documentation | Android Application

Project Proposal

Project Overview

Eco-Guardian is a community-driven Android application that empowers citizens to report local environmental issues — such as illegal dumping, pollution, and litter — with minimal effort. Users take a photo, describe what they see, and the app's built-in AI generates a professionally structured report ready to be submitted to local authorities. Every submitted report is pinned to a live community map, giving neighbourhoods a shared, real-time view of environmental health in their area.

Problem Statement

Citizens who witness environmental violations have no simple way to formally report them. Existing reporting channels are slow, bureaucratic, and require users to know which authority to contact and how to write a formal complaint. As a result, most issues go unreported and unresolved.

Proposed Solution

Eco-Guardian removes every barrier between seeing a problem and reporting it. The AI-powered report generation is the core differentiator — unlike all existing civic reporting apps, Eco-Guardian writes the formal complaint for the user. The community map adds collective visibility, turning individual actions into shared environmental awareness.

Objectives

- Enable citizens to report environmental issues in under two minutes.
- Use AI to generate structured, authority-ready reports from user descriptions.
- Reduce friction of formal complaint processes through automation.
- Build community awareness through a shared, real-time environmental map.
- Provide local authorities with consistently formatted, geotagged reports.

Project Scope

Platform	Android (Kotlin)
Target Version	Android 8.0 (API 26) and above
Backend	Firebase (Auth, Firestore, Storage, Cloud Functions)
AI Integration	LLM API via Cloud Function — key never stored in APK
Map	Google Maps SDK
Duration	~5 months (February – July 2026)
Team Size	4 members

Core Features (MVP)

- Photo capture or gallery selection with a description form.
- AI-generated authority report draft — editable before submission.
- AI issue categorisation (litter, pollution, illegal dumping, etc.).
- Community map showing all reported issues as geotagged pins.
- Report feed — filterable list of community submissions.
- Runtime permissions management for camera and location access.

Additional Features (if capacity allows)

- Community clean-up event listings (read-only).
- User profile showing submitted reports and their status.
- Notification and home-location preferences via DataStore.

Team & Responsibilities

Member	Role	Owns
Dev 1	Tech Lead	Architecture, NavGraph, Data Layer, Code Review
Dev 2	Android UI	Screen Implementation, RecyclerView / Compose UI
Dev 3	Backend Glue	API Integration, Repository Layer, DataStore
Dev 4	QA & Permissions	Runtime Permissions Flow, Manual Testing, Edge Cases

Key Performance Indicators (KPIs)

- Report submission success rate: > 95%
- AI draft generation time: < 5 seconds
- Community map load time: < 3 seconds
- Crash-free session rate: > 99%
- Permission grant rate (first prompt): > 80%

Literature Review

Awaiting Lecturer Feedback

This section will be completed following the lecturer's review and assessment of the project concept. Content will include an evaluation of existing environmental reporting platforms, a gap analysis identifying the opportunity Eco-Guardian addresses, and references to relevant academic and industry sources.

Pending Lecturer feedback and evaluation to be added here after submission review.

Requirements Gathering

3.1 Stakeholder Analysis

The following stakeholders have been identified as the key parties who interact with or are affected by the Eco-Guardian system.

Stakeholder	Need	How App Serves Them	Acceptance Criteria
Citizens (Reporters)	A fast, low-effort way to report environmental issues without bureaucracy.	Photo + description → AI writes the report in under 2 minutes.	Report submitted successfully; AI draft generated in < 5s.

Local Authorities	Consistent, well-formatted reports with location data they can act on.	Receive structured, geotagged reports with clear issue category and description.	Reports include location, category, photo, and formatted description.
Community Members	Awareness of environmental issues in their neighbourhood.	Community map shows all reported issues with status and category.	Map loads with correct pins in < 3 seconds.
Development Team	Clear requirements, manageable scope, clean codebase.	MVVM architecture, modular ownership, Firebase for backend.	All PRs reviewed before merge; no single-person feature ownership.

3.2 User Stories

As a Citizen (Reporter)

- As a citizen, I want to take a photo of an environmental issue so that I can report it without writing a formal letter.
- As a citizen, I want the app to generate a professional report from my description so that I don't need to know how to write to authorities.
- As a citizen, I want to review and edit the AI draft before submitting so that I feel in control of what is sent.
- As a citizen, I want to see my report appear on the community map after submission so that I know it was received.

As a Community Member (Viewer)

- As a community member, I want to see a map of all reported issues so that I can understand the environmental health of my neighbourhood.
- As a community member, I want to filter the report feed by category so that I can focus on the issues I care about most.
- As a community member, I want to tap a map pin and see the full report detail so that I can learn more about a specific issue.

3.3 Functional Requirements

Authentication

1. FR-01: The system shall allow users to register with an email address and password.
2. FR-02: The system shall allow registered users to log in and log out.
3. FR-03: The system shall persist the user session across app restarts.

Report Submission

1. FR-04: The system shall allow users to capture a photo using the device camera.
2. FR-05: The system shall allow users to select a photo from the device gallery.
3. FR-06: The system shall allow users to enter a free-text description of the issue.
4. FR-07: The system shall automatically detect and attach the user's current GPS location to the report.
5. FR-08: The system shall send the photo and description to an AI service and return a structured report draft.
6. FR-09: The system shall display the AI-generated draft in an editable text field before submission.
7. FR-10: The system shall categorise the report automatically (e.g. litter, pollution, illegal dumping).
8. FR-11: The system shall submit the finalised report to Firestore upon user confirmation.

Community Map

1. FR-12: The system shall display a map showing all submitted reports as geotagged pins.
2. FR-13: The system shall colour-code map pins by issue category.
3. FR-14: The system shall allow users to tap a pin to view the report detail screen.
4. FR-15: The system shall centre the map on the user's current location on launch.

Report Feed

1. FR-16: The system shall display a scrollable list of all community reports.
2. FR-17: The system shall support multiple item types in the feed (report cards, section headers, loading indicators).
3. FR-18: The system shall allow users to filter the feed by issue category.

Permissions & Settings

1. FR-19: The system shall request camera permission at runtime before accessing the camera.
2. FR-20: The system shall request location permission at runtime before attaching GPS coordinates.
3. FR-21: The system shall handle denied and permanently-denied permission states with clear rationale screens.
4. FR-22: The system shall allow users to set a Home location that personalises the map and feed default view.
5. FR-23: The system shall store user preferences using Preferences DataStore.

3.4 Non-Functional Requirements

Category	Requirement	Detail
Performance	Response time	Community map loads in < 3 seconds. AI draft returned in < 5 seconds.
Performance	List smoothness	Report feed scrolls at 60 fps with no dropped frames on mid-range devices.
Security	API key protection	LLM API key is stored server-side in Cloud Functions only — never bundled in the APK.
Security	Authentication	All Firestore reads and writes require an authenticated user session via Firebase Auth.
Security	Permissions	Camera and location are requested at runtime. No permission is assumed granted.
Usability	Compatibility	App supports Android 8.0 (API 26) and above, covering over 95% of active Android devices.
Usability	Accessibility	All interactive elements meet minimum 48dp touch target size. Content descriptions on all images.
Reliability	Offline handling	If network is unavailable during submission, the app shows a clear error and does not lose user input.
Reliability	AI fallback	If AI generation fails or times out (10s), the user can submit their raw description directly.
Maintainability	Architecture	MVVM + Clean Architecture with strict separation of UI, Domain, and Data layers.
Maintainability	Code standards	All code follows Kotlin coding conventions. No feature is owned by a single developer.

System Analysis & Design

4.1 Software Architecture

Architecture Style: MVVM + Clean Architecture (Layered)

Three distinct layers to separate concerns and ensure testability:

- **Presentation Layer** — Jetpack Compose UI + ViewModels
- **Domain Layer** — Use Cases + Domain Models
- **Data Layer** — Repositories, Remote Data Source , Local Data Source (Room)

Technology Stack

Layer	Technology
Mobile Language	Kotlin
UI Framework	Jetpack Compose
Architecture Pattern	MVVM + Clean Architecture
Maps	Google Maps SDK for Android
AI Integration	Anthropic Claude API (via Firebase Cloud Functions)
Authentication	Firebase Authentication / Supabase Auth (GoTrue)
Database (Remote)	Cloud Firestore / Supabase
File Storage	Firebase Storage / Supabase Storage
Push Notifications	Firebase Cloud Messaging (FCM)
Networking	Retrofit 2 + OkHttp
Image Loading	Coil
Local Database	Room (offline drafts)
Dependency Injection	Hilt / Coin
Backend Logic	Firebase / Supabase

4.2 System Analysis & Design Diagrams

All Diagrams